

vLLM专题（三）-快速开始

原创 AI专题精讲 已于 2025-03-19 11:31:21 修改 阅读量219 收藏 点赞数 版权

分类专栏: 大模型专题系列 文章标签: 人工智能



大模型专题系列 专栏收录该内容

¥49.90
¥99.00

111 篇文章

已订阅

8折续

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益

本指南将帮助您快速开始使用 vLLM 执行：

- 离线批量推理
- 使用 OpenAI 兼容服务器进行在线服务

1. 先决条件

操作系统：Linux

Python：3.9 – 3.12

2. 安装

如果您使用的是 NVIDIA GPU，您可以直接使用 pip 安装 vLLM。

建议使用 uv，一个非常快速的 Python 环境管理器，来创建和管理 Python 环境。请按照文档安装 uv。
安装 uv 后，您可以创建一个新的 Python 环境，并使用以下命令安装 vLLM：

```
uv venv myenv --python 3.12 --seed
source myenv/bin/activate
uv pip install vllm
```

您也可以使用 conda 来创建和管理 Python 环境。

```
conda create -n myenv python=3.12 -y
conda activate myenv
pip install vllm
```

注意

对于非 CUDA 平台，请参考此处的具体说明以了解如何安装 vLLM。

3. 离线批量推理

安装 vLLM 后，您可以开始为一组输入提示生成文本（即离线批量推理）。请参见示例脚本：[examples/offline_inference/basic.py](https://github.com/vllm-project/vllm/blob/main/examples/offline_inference/basic.py)

此示例的第一行导入了 `LLM` 和 `SamplingParams` 类：

- `LLM` 是用于通过 vLLM 引擎运行离线推理的主要类。
- `SamplingParams` 指定了采样过程的参数。

```
from vllm import LLM, SamplingParams
```

下一部分定义了一组输入提示和用于文本生成的采样参数。采样温度设置为 0.8，核采样概率设置为 0.95。您可以在[此处](#)找到有关采样参数的更多信息。

```
prompts = [
    "Hello, my name is",
    "The president of the United States is",
    "The capital of France is",
    "The future of AI is",
]
sampling_params = SamplingParams(temperature=0.8, top_p=0.95)
```

`LLM` 类初始化 vLLM 引擎和 OPT-125M 模型用于离线推理。支持的模型列表可以在[此处](#)找到。

```
llm = LLM(model="facebook/opt-125m")
```

注意

默认情况下，vLLM 从 HuggingFace 下载模型。如果您希望使用来自 ModelScope 的模型，请在初始化引擎之前设置环境变量 `VLLM_USE_MODELSCOPE`。

现在，进入有趣的部分！输出是通过 `llm.generate` 生成的。它将输入提示添加到 vLLM 引擎的等待队列，并执行 vLLM 引擎以高吞吐量生成输出。输出作为 `RequestOutput` 对象的列表返回，其中包含所有输出的标记。

```
outputs = llm.generate(prompts, sampling_params)

for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(f"Prompt: {prompt!r}, Generated text: {generated_text!r}")
```

4. OpenAI 兼容服务器

vLLM 可以[作为实现 OpenAI API 协议的服务器进行部署](#)。这使得 vLLM 可以作为 OpenAI API 的替代品，供使用 OpenAI API 的应用程序使用。默认情况下，它在 `http://localhost:8000` 启动服务器。您可以使用 `--host` 和 `--port` 参数指定地址。当前，服务器一次只能托管一个模型，并实现了诸如列出模型、创建聊天完成和创建完成等端点。

运行以下命令以启动 vLLM 服务器，并使用 Qwen2.5-1.5B-Instruct 模型：

```
vllm serve Qwen/Qwen2.5-1.5B-Instruct
```

注意

默认情况下，服务器使用存储在分词器中的预定义聊天模板。您可以在[这里](#)了解如何覆盖它。

该服务器可以使用与 [OpenAI API 相同的格式进行查询](#)。例如，要列出模型：

```
curl http://localhost:8000/v1/models
```

您可以传入参数 `--api-key` 或环境变量 `VLLM_API_KEY`，以启用服务器在请求头中检查 API 密钥。

4.1 使用 vLLM 的 OpenAI 完成 API

一旦服务器启动，您可以使用输入提示查询模型：

```
curl http://localhost:8000/v1/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "Qwen/Qwen2.5-1.5B-Instruct",
    "prompt": "San Francisco is a",
    "max_tokens": 7,
    "temperature": 0
  }'
```

由于该服务器与 OpenAI API 兼容，您可以将其作为任何使用 OpenAI API 的应用程序的替代品。例如，查询服务器的另一种方式是通过 `openai` Python 包：

```

from openai import OpenAI

# Modify OpenAI's API key and API base to use vLLM's API server.
openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"
client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)
completion = client.completions.create(model="Qwen/Qwen2.5-1.5B-Instruct",
                                       prompt="San Francisco is a")
print("Completion result:", completion)

```

更详细的客户端示例可以在这里找到: [examples/online_serving/openai_completion_client.py](https://github.com/vllm-project/vllm/blob/main/examples/online_serving/openai_completion_client.py)

4.2 使用 vLLM 的 OpenAI 聊天完成 API

vLLM 还支持 OpenAI 聊天完成 API。聊天界面是一种更动态、互动的与模型沟通的方式，允许进行往返交流，并可以将对话历史存储。这对于需要上下文或更详细解释的任务非常有用。

您可以使用创建聊天完成端点与模型进行交互：

```

curl http://localhost:8000/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "Qwen/Qwen2.5-1.5B-Instruct",
    "messages": [
      {"role": "system", "content": "You are a helpful assistant."},
      {"role": "user", "content": "Who won the world series in 2020?"}
    ]
  }'

```

另外，您也可以使用 openai Python 包：

```
from openai import OpenAI
# Set OpenAI's API key and API base to use vLLM's API server.
openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)

chat_response = client.chat.completions.create(
    model="Qwen/Qwen2.5-1.5B-Instruct",
    messages=[
        {
            "role": "system", "content": "You are a helpful assistant."},
        {
            "role": "user", "content": "Tell me a joke."},
    ]
)
print("Chat response:", chat_response)
```



AI专题精讲

已关注



分享



专栏目录

已订阅